Information Retrieval Assignment Report-HW7

This assignment focuses on building a machine learning model for **binary email classification**, where emails are categorized as either:

- **Spam** (unwanted or malicious email)
- **Ham** (legitimate email)

The classifier was developed using:

- **TF-IDF vectorization** for feature extraction
- **Logistic Regression** for classification
- **5-fold cross-validation** for evaluation

The objective was to train an accurate classifier capable of distinguishing spam from ham using textual features extracted from email content.

Dataset Description

The dataset used in this assignment was derived from the **TREC 2007 Spam Track**.

Instead of using the original raw .eml email files, a **pre-processed CSV version** of the dataset was used.

Dataset Statistics

Property	Value
Source	TREC 2007 Spam Track (Pre-processed)
File	processed_data.csv
File Size	338 MB
Total Emails	75,419
Spam Emails	50,199 (66.6%)
Ham Emails	25,220 (33.4%)

Reason for Using Pre-processed Data

The original raw email files were unavailable.

The processed_data.csv we downloaded from https://www.kaggle.com/datasets/imdeepmind/preprocessed-trec-2007-public-corpus-dataset

already contained extracted fields necessary for classification, including:

- Email subject
- Sender information
- Recipient information
- Email body text
- Label

This allowed the focus of the assignment to remain on machine learning classification rather than raw email parsing.

Dataset Fields

Field	Description
label	1 = Spam, 0 = Ham
subject	Email subject line
email_to	Recipient address
email_from	Sender address
message	Email body content

Missing Data

The dataset contained some missing values:

Field	Missing Values
Subject	793
Email To	576
Messa	1,487

ge	

All missing values were replaced with empty strings during preprocessing.

Methodology

The classification pipeline consisted of preprocessing, feature extraction, model training, and evaluation.

Text Preprocessing

The following preprocessing steps were applied:

1. Combined all textual fields into one text representation:

subject + email_to + email_from + message

2. Replaced missing values with empty strings
3. Converted text to lowercase
4. Removed English stopwords

This produced a unified clean text representation for each email.

Feature Extraction using TF-IDF

The cleaned email text was converted into numerical features using **Term Frequency-Inverse Document Frequency (TF-IDF)**.

TF-IDF captures the importance of words based on:

- How frequently they appear in an email
- How rare they are across the dataset

TF-IDF Parameters

Parameter	Value
Max Features	5,000
Min Document Frequency	2
Max Document Frequency	95%
Stop Words	English

These settings reduce noise while preserving informative features.

Classification Model

The classifier used was **Logistic Regression**.

This model was selected because it:

- Performs well for binary classification
- Trains efficiently on large text datasets
- Produces interpretable feature weights

The learned coefficients help identify which words indicate spam or ham.

Training Configuration

Parameter	Value
Train/Test Split	80% / 20%
Training Emails	60,335
Test Emails	15,084
Cross Validation	5-fold
Random Seed	42

The fixed random seed ensured reproducibility.

Results

The classifier achieved excellent performance.

Overall Accuracy

Metric	Score
Test Accuracy	**99.66%**
Cross Validation Accuracy	**99.53% ± 0.19%**

These results indicate highly consistent performance.

Confusion Matrix

Actual / Predicted	Ham	Spam
Ham	4,999	45
Spam	7	10,033

Interpretation

The model correctly classified:

- **4,999 ham emails**
- **10,033 spam emails**

Misclassifications:

- **45 false positives**
- **7 false negatives**

This demonstrates strong classification accuracy.

Classification Report

Class	Precision	Recall	F1-score
Ham	1.00	0.99	0.99
Spam	1.00	1.00	1.00

The near-perfect scores indicate excellent discrimination between classes.

Feature Analysis

Logistic Regression coefficients reveal which words strongly predict spam or ham.

Top Spam Indicators

Rank	Word	Coefficient
1	uwaterloo	+8.92
2	ca	+6.79
3	plg	+5.51
4	producttestpanel	+4.64
5	content	+3.96
6	type	+3.64
7	price	+3.15
8	life	+2.99
9	pills	+2.84
10	transfer	+2.68

These words commonly appear in:

- Promotional spam
- Financial scams
- Pharmaceutical advertisements

Top Ham Indicators

Rank	Word	Coefficient
1	org	-7.79
2	perl	-7.43
3	samba	-6.61
4	wrote	-6.59
5	cnn	-5.55
6	debian	-5.32
7	lists	-5.30
8	cbs	-4.86
9	bbc	-4.14
10	unsubscribe	-3.78

These terms are associated with:

- Technical discussions
- News communication
- Legitimate mailing lists

Analysis

The classifier performed exceptionally well for several reasons.

Clear Class Separation

Spam and ham emails contain highly distinct vocabularies.

Spam frequently contains:

- Product advertisements
- Financial terms
- Suspicious domains

Ham often contains:

- Technical discussions
- Organization domains
- Legitimate correspondence patterns

This strong separation improves classification accuracy.

Effectiveness of TF-IDF

TF-IDF effectively emphasizes discriminative terms while down-weighting common words.

Words like **pills**, **price**, **transfer**

receive strong spam weights.

Words like: **Perl, org, bbc**

strongly indicate ham.

Low Error Rate

Only **52 misclassifications** occurred out of **15,084 test emails**.

This is an error rate of approximately **0.34%**.

The low false negative count is particularly important because it means very few spam emails escaped detection.

Limitations

The primary limitation is that the original raw email files were unavailable.

Using pre-processed data means:

- Header parsing was already completed
- Raw MIME structure was unavailable
- Certain metadata features could not be explored

However, all required fields for classification were present.

Conclusion

The spam/ham email classifier achieved **99.66% test accuracy**, demonstrating that Logistic Regression combined with TF-IDF is highly effective for spam detection.

Although pre-processed data was used, the classification task was successfully completed.

The results confirm that classical machine learning approaches remain highly effective for text classification.